CASE STUDY REPORT

Smart Agriculture and Automated Irrigation System

Name – Kirti Khumendra Bisen
UID- 25103013
Date- 25 oct 2026

1. Problem Statement

A smart agriculture system collects soil-moisture readings, temperature measurements, weather information, and water-level data. It processes the sensor readings, generates irrigation recommendations, and controls pumps and monitoring devices. Several tasks compete for CPU time, memory, and shared control resources. Without proper operating-system management, sensor data may be lost or corrupted, irrigation decisions may be delayed, and tasks may wait indefinitely for resources.

The problem is to design and study an operating-system-based approach that schedules tasks efficiently, synchronizes shared sensor data, avoids unsafe resource allocation, and allocates limited memory effectively.

2. Problem Analysis

The system contains periodic sensor-reading tasks and event-driven tasks for data analysis, alerts, irrigation decisions, and pump control. Sensor tasks produce data while an analysis task consumes it through a finite queue. Multiple tasks may access shared data or request resources such as communication channels, memory workspaces, and the pump-control interface. The CPU and memory are limited, so poor scheduling can increase waiting time, unsynchronized access can cause race conditions, and unsuitable resource allocation can lead to deadlock or allocation failure.

Constraints and assumptions: the case study models one CPU; example arrival and burst times are known; resource quantities are represented as non-negative integers; the sensor queue has a fixed capacity; and input readings are assumed to be validated. Physical hardware failures, network delays, true parallel execution, and real-time certification are outside this educational model.

3. Objectives

- Schedule sensor acquisition, analysis, alerts, and irrigation tasks and calculate completion, turnaround, waiting, and response times.
- Protect the shared sensor queue with Producer–Consumer synchronization.
- Use Banker's Algorithm to determine whether a resource-allocation state is safe.
- Compare First Fit and Best Fit for allocating memory to sensor buffers and processing tasks.
- Integrate the four OS concepts into a single smart-irrigation workflow and define tests for validating the simulation.

4. Operating-System Concepts and Algorithm Design

4.1 CPU Scheduling — FCFS

First Come, First Served executes ready tasks in arrival order. It is simple and provides a baseline, although a long task may delay an urgent irrigation-control task (convoy effect).

Pseudocode: Sort tasks by arrival time; set time = 0. For each task, if time is less than arrival time, advance time to arrival. Record start; add burst time to obtain completion time. Calculate TAT = CT − AT, WT = TAT − BT, and RT = first start − AT.

4.2 Process Synchronization — Producer–Consumer

Sensor tasks are producers and the analysis task is the consumer of a bounded queue. The mutex protects queue operations; empty counts free slots and full counts stored readings. Producer: wait(empty), wait(mutex), insert reading, signal(mutex), signal(full). Consumer: wait(full), wait(mutex), remove reading, signal(mutex), signal(empty). This prevents race conditions and handles full/empty conditions.

4.3 Deadlock Avoidance — Banker's Algorithm

Tasks may request resources such as a communication channel, analysis workspace, or pump-control interface. Need = Maximum − Allocation. The safety algorithm repeatedly finds an unfinished task whose Need is no greater than Work, marks it complete, and adds its allocated resources to Work. The state is safe if every task can finish in some order.

4.4 Memory Management — First Fit and Best Fit

First Fit selects the first free block large enough for a request; Best Fit selects the smallest suitable block. With variable-sized blocks, the chosen block is split and any remainder stays free. Adjacent free blocks should be merged when released. Small separated free spaces can cause external fragmentation.

5. Algorithm Worked Examples and System Workflow

5.1 Workflow diagram

SENSORS Soil moisture • Temperature • Weather • Water level
BOUNDED SENSOR QUEUE Producer–Consumer • mutex / empty / full
CPU SCHEDULER Acquisition • Analysis • Alerts • Irrigation decision
RESOURCE & MEMORY MANAGEMENT Banker's safety check • First Fit / Best Fit
OUTPUT Irrigation recommendation → safeguarded pump/valve control

Workflow: Sensors → synchronized queue → scheduled processing → resource and memory checks → irrigation recommendation / pump control.

5.2 FCFS calculation

Let T1(AT=0, BT=4), T2(AT=0, BT=3), and T3(AT=0, BT=2). FCFS order: T1 → T2 → T3. Completion times = 4, 7, 9; turnaround times = 4, 7, 9; waiting times = 0, 4, 7. Average WT = (0+4+7)/3 = 3.67 units. Average TAT = (4+7+9)/3 = 6.67 units.

Process	CT	TAT	WT
P1	4	4	0
P2	7	7	4
P3	9	9	7

5.3 Banker's Algorithm calculation

For the standard five-process, three-resource example, Available = (3,3,2). After calculating Need = Maximum − Allocation, one safe sequence is P1 → P3 → P4 → P0 → P2. Initially, P1 has Need (1,2,2), which is no greater than Available (3,3,2), so P1 can finish and release its allocation. This illustrates how completed tasks release resources for the remaining tasks.

5.4 Memory allocation calculation

Free blocks = 100, 500, 200, 300 KB; request = 180 KB. First Fit selects 500 KB and leaves 320 KB. Best Fit selects 200 KB and leaves 20 KB.

6. Test Cases and Execution Results

The following are expected results derived from the examples. Enter actual results only after executing each test in the implemented mini-project.

Test	Input	Expected result
FCFS	T1(0,4), T2(0,3), T3(0,2)	Avg WT 3.67; Avg TAT 6.67
CPU idle	AT=3, BT=2	Idle 0–3; CT=5; WT=0; TAT=2
Storage full	Capacity 2; produce 3 items	Count stays 2; no overflow
Storage empty	Consume with no items	Request blocked/rejected; count 0
Banker safe	Available (3,3,2), standard example	Safe sequence found
Banker unsafe	Unsafe state/request example	Unsafe state detected or request denied
First Fit	Blocks 100,500,200,300 KB; request 180 KB	500 KB selected; 320 KB remains
Best Fit	Same blocks; request 180 KB	200 KB selected; 20 KB remains
Invalid input	Negative burst or Allocation > Maximum	Input rejected with message

7. Integration into the Smart Agriculture System

Sensor tasks place moisture, temperature, weather, and water-level readings into the synchronized queue. The CPU scheduler runs data acquisition, filtering, threshold

evaluation, alerts, and irrigation decisions. Banker's Algorithm checks whether resource allocation remains safe, while First Fit or Best Fit allocates memory for sensor batches and temporary analysis data. If soil moisture is below a configured threshold and water is available, the system can recommend irrigation. A real pump must also have independent water-level limits, manual override, fault detection, and electrical safeguards.

8. Conclusion

The case study demonstrates how CPU scheduling, Producer–Consumer synchronization, Banker's Algorithm, and First Fit/Best Fit memory allocation can support a smart irrigation controller. These concepts help organize competing tasks, protect shared sensor readings, avoid unsafe resource states, and manage limited memory. The calculations and test cases provide a basis for validating the mini-project. The model remains educational and does not replace hardware testing or a certified real-time control design.

References

- Silberschatz, A., Galvin, P. B., and Gagne, G., Operating System Concepts, Wiley.
- Tanenbaum, A. S., and Bos, H., Modern Operating Systems, Pearson.
- Operating Systems course notes and lecture material provided by the instructor.

Student Details and Declaration

I declare that this case study is submitted for academic evaluation. Execution results will be recorded only after the corresponding tests have been performed.

Name – Kirti Kumendra Bisen
UID- 25103013
Sign -